

# Les bases de l'animations

Les représentations graphiques font partie des tâches fondamentales d'un physicien, d'un mathématicien, d'un data scientist... bref, de tout scientifique. Elles permettent de mieux comprendre et d'interpréter les résultats d'un projet scientifique. Pour une présentation encore plus fluide, dynamique et engageante, les animations constituent un atout précieux. Dans ce notebook, nous allons voir comment créer des animations à l'aide de la bibliothèque Python matplotlib (Mathematics Plotting Library).

## C'est quoi une animation?

Une animation est composée d'une suite de frames (images fixes), chacune représentant une figure différente. Enchaînées rapidement ces frames donnent l'illusion du mouvement, d'une animation. Avec matplotlib, les frames sont des figures, des représentations graphiques. Alors comment se fait l'animation avec matplotlib?

## Matplotlib

C'est la librairie de visualisation la plus utilisée de Python pour faire des représentations en deux dimensions des tableaux NumPy

## Matplotlib.animation.FuncAnimation

La classe matplotlib.animation.FuncAnimation est utilisée pour faire de l'animation par appel répétitif d'une même fonction qui met à jour nos données.

Les paramètres (les plus importants) pris par l'objet FuncAnimation sont les suivants:

- *fig*: La figure que l'on veut animer
- *func*: La fonction qui vas mettre a jour a chaque fois les données de notre figure
- *frames*: La durée de l'animation. il determine de le nombre d'images fixe qui vont etre enchainées pour creer le mouvenment
- *interval*: Determine le temps (en milliseconde) de passage d'une frame à une autre.

## Les differentes étapes pour creer une animation avec FuncAnimation

1- Construire une figure initial vide et l'enregistrer dans une variable

2- Creer une fonction pour l'animation.

3- Creer une animation en passant la Figure et la fonction d'animation à l'objet FuncAnimation et l'assigner à une variable.

4- Sauvegarder l'animation en utilisant l'une des méthodes suivantes:

- `plt.show` pour afficher l'animation dans une fenetre
- `Animation.to_html5_video` pour creer un tag video HTML
- `Animation.to_jshtml` to create HTML code with interactive JavaScript animation controls
- `Animation.save` to sauvegarder l'animation dans un fichier

### Exemple 1: animation d'une seule courbe

Animation de la représentation de la fonction cosinus au cours du temps.

```
In [27]: import matplotlib.pyplot as plt
import numpy as np
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# 1.1- Creation de la figure et de l'axe
fig, ax = plt.subplots()

#1.2- creation du tableau de points sur l'axe des abscisses
t = np.linspace(0,2*np.pi,100)

# 1.3- Fonction de la courbe cosinus
def cosinus(time):
    return np.cos(time)

#1.4- Calcul des points sur l'axe des ordonnées
y = cosinus(t)
#1.5- Creation d'un objet figure de type Line2D

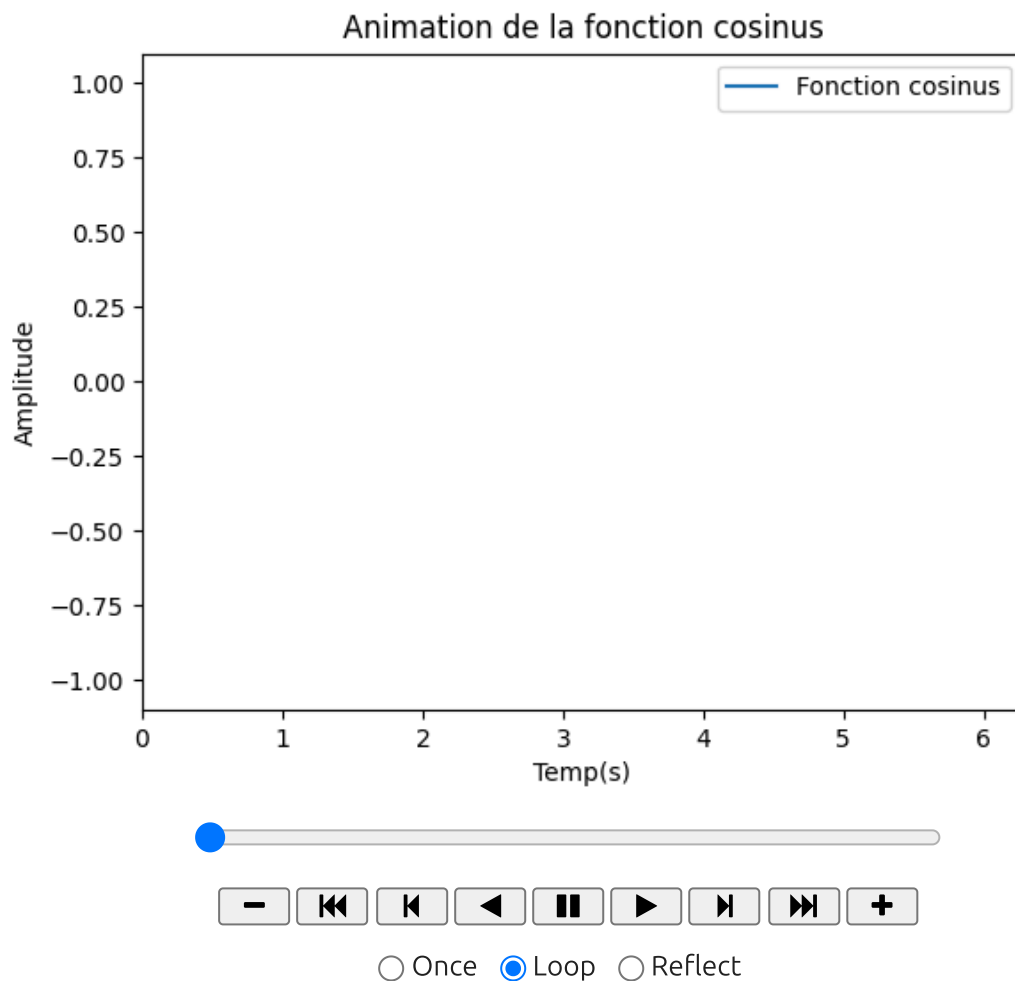
line, = ax.plot(t[0],y[0], label = 'Fonction cosinus') # Cette écriture
ax.set_xlim(0,2*np.pi)
ax.set_ylim(-1.1, 1.1)
ax.set_xlabel('Temp(s)')
ax.set_ylabel('Amplitude')
ax.set_title('Animation de la fonction cosinus')
ax.legend()

#2- Création de la fonction de mise à jour des données des figures
def updatator(frame):
    time = t[:frame]
    y_cos= y[:frame]
    line.set_xdata(time)
    line.set_ydata(y_cos)
    return line

#3- Création de l'animation
anim = FuncAnimation(fig = fig, func = updatator, frames=100, interval=100)

video = anim.to_jshtml()
html = HTML(video)
display(html)

plt.close(fig) # Ferme la figure pour éviter l'affichage en double dans
```



### Exemple 2: animation de deux courbes

Nous allons à présent animer deux projectiles dans un champ de pesanteurs  $g$  lancé à des valeurs de vitesse différentes.

```
In [31]: fig, ax = plt.subplots()

t = np.linspace(0,20, 500)

def altitude(temps, v0, g=9.81):
    return -0.5*g*temps**2+v0*temps

pos_1 = altitude(t, 5)
pos_2 = altitude(t, 7)

scat = plt.scatter(t[0], pos_1[0], label=f'Projecctile lancée à la vitess
line, = plt.plot(t[0], pos_2[0], label=f'Projecctile lancée à la vitesse
ax.set_xlim(0,2)
ax.set_ylim(0, max(pos_2)+0.2)
ax.set_xlabel('temps (s)')
ax.set_ylabel('Altitude (m)')
ax.set_title('Animation de la chute libre de deux projectiles')
ax.legend()

def update(frame):
    time = t[:frame]
    Pos_1 = pos_1[:frame]
    Pos_2 = pos_2[:frame]
```

```

# mise à jour des lines plots
data = np.stack([time, Pos_1]).T
line.set_xdata(time)
line.set_ydata(Pos_2)
# mise à jour des scatters plots
scat.set_offsets(data)
return (line,scat)

animation = FuncAnimation(fig = fig, func = update, frames = 100, interval=100)
video = animation.to_html5_video()
html = HTML(video)
display(html)
plt.close()

```



Notons que la méthode `set_data()` ou (`set_xdata()`, `set_ydata()`) n'est utilisée que lorsque notre graphe est une ligne continue. S'il s'agit de pointillé, une autre méthode sera utilisée. Comme vous l'aurez remarqué dans l'exemple précédent, la méthode `set_offsets` a été utilisée. pour les autres types de graphes, cliquez sur ce lien [animations.html](#)

### Exemple 3: Animation d'un seul point

Animons un projectile de masse  $m$  lancé à la vitesse  $v_0 = 3m/s$  sous un angle  $\theta$

Les coordonnées du vecteur vitesse sont les suivantes:  $v_x = v_0 \cos(\theta)$ ,  
 $v_y = v_0 \sin(\theta) - gt$

Les coordonnées du vecteur position sont :  $x = v_0 \cos(\theta)t$ ,  $y = v_0 \sin(\theta)t - \frac{1}{2}gt^2$

```
In [33]: fig, ax = plt.subplots()

pos, = ax.plot(0,0,'o')
ax.set_xlim(0,1)
ax.set_ylim(-1,1)

v0 = 3.
theta = 60./180. * np.pi
g = 9.8

def pos_x(t):
    return v0 * np.cos(theta) * t

def pos_y(t):
    return v0 * np.sin(theta) * t - g*t**2/2.

dt = 0.01

l1x = [0]
l1y = [0]

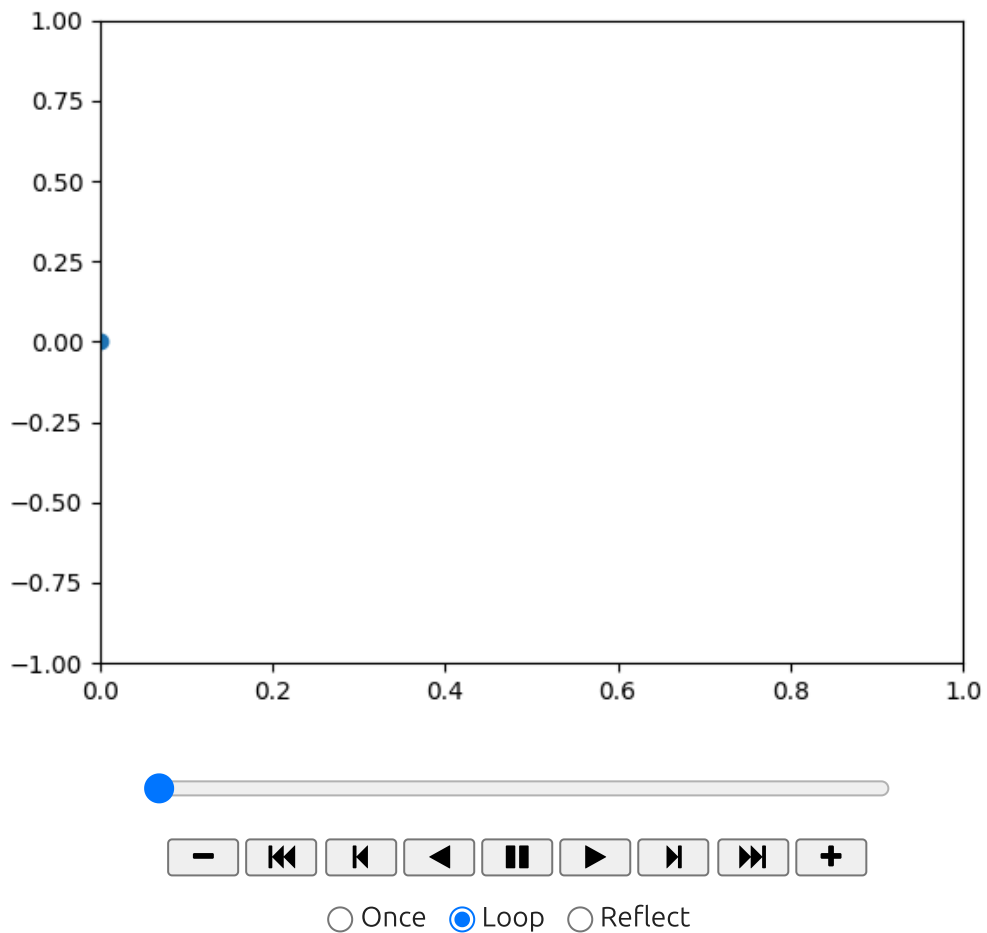
total_frames = 70

def animate(frame):
    tx, ty = pos_x(frame*dt), pos_y(frame*dt)
    pos.set_data([tx], [ty]) # Wrap in lists to make them sequences
    if (len(l1x) < total_frames):
        l1x.append(tx)
    if (len(l1y) < total_frames):
        l1y.append(ty)

    return pos # Return the artists that were modified

anim = FuncAnimation(
    fig=fig, func=animate, frames = total_frames, interval=30)
plt.close()
HTML(anim.to_jshtml())
```

Out[33]:

**Exemple 4: Animation d'un seul point suivi par une ligne dans son déplacement**

Exemple similaire à celui de l'exemple 3.

```
In [35]: fig, ax = plt.subplots()

pos, = ax.plot(0,0,'o')
ax.set_xlim(0,1)
ax.set_ylim(-1,1)

v0 = 3.
theta = 60./180. * np.pi
g = 9.8

def pos_x(t):
    return v0 * np.cos(theta) * t

def pos_y(t):
    return v0 * np.sin(theta) * t - g*t**2/2.

dt = 0.01

l1x = [0]
l1y = [0]
line, = ax.plot(l1x,l1y)

total_frames = 70

def animate(frame):
```

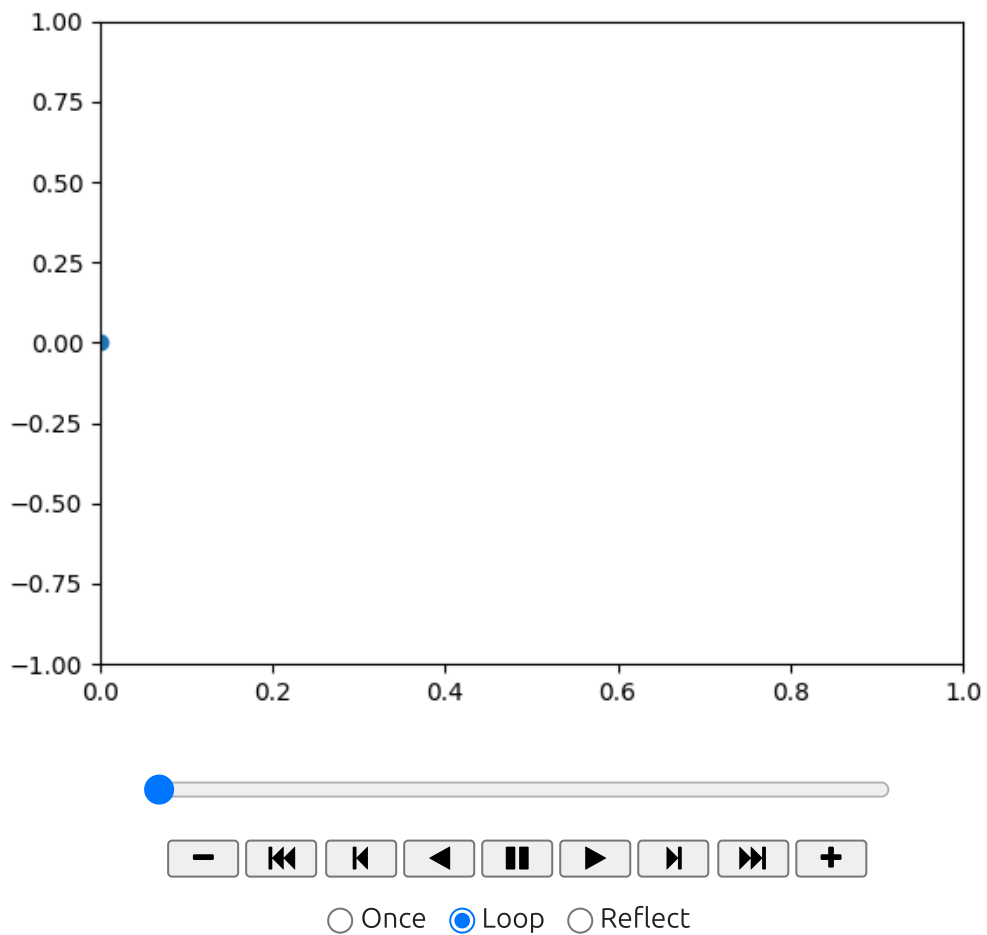
```

tx, ty = pos_x(frame*dt), pos_y(frame*dt)
pos.set_data([tx], [ty]) # Wrap in lists to make them sequences
if (len(l1x) < total_frames):
    l1x.append(tx)
if (len(l1y) < total_frames):
    l1y.append(ty)
line.set_data(l1x, l1y)
return pos, line # Return the artists that were modified

anim = FuncAnimation(
    fig=fig, func=animate, frames = total_frames, interval=30)
plt.close()
HTML(anim.to_jshtml())

```

Out[35]:



In [ ]: